# Analisis dan Implementasi Algoritma Bellman-Ford untuk Jalur Terpendek pada Graf Berbobot Negatif

*Laporan Tugas Analisis Algoritma, Mata Kuliah Algoritma Komputasi*

Muh Fauzi Ramadhan Nugraha
*Magister Teknik Informatika, Departemen Teknik Informatika*
*Fakultas Teknik, Universitas Hasanuddin*
Gowa, Indonesia
NIM D082261017

***Abstrak*—Algoritma Bellman-Ford menghitung jarak terpendek dari satu simpul sumber pada graf berarah yang boleh berbobot negatif, dan mendeteksi siklus negatif yang terjangkau dari sumber. Laporan ini menganalisis kebenaran, kompleksitas, dan perilaku empiris algoritma tersebut. Analisis memuat pseudocode, contoh perhitungan pada graf empat simpul, bukti kebenaran jarak setelah $|V| - 1$ putaran, bukti deteksi siklus negatif, dan penurunan kompleksitas: $\Theta(|V| \cdot |E|)$ pada kasus terburuk, $\Theta(|V| + |E|)$ pada kasus terbaik dengan penghentian dini, dan ruang tambahan $\Theta(|V|)$. Algoritma diimplementasikan dari nol dalam Python, diuji pada tiga kasus kecil (rute kurir, segitiga tukar uang, dan rantai dengan urutan terburuk) dan studi kasus distribusi barang, serta dicocokkan dengan scipy pada 1.000 graf acak tanpa selisih. Eksperimen memakai tiga keluarga graf dengan $|E| = 4|V|$, enam ukuran, dan lima ulangan per ukuran. Pangkat pertumbuhan waktu terhadap $|V|$ adalah 1,94 pada kasus terburuk (teori 2) dan 1,03 pada kasus terbaik (teori 1). Pada graf acak jumlah putaran tumbuh dari 8,2 menjadi 15,8 mengikuti $\log|V|$, tetapi waktu per pemeriksaan sisi naik sekitar 4 kali pada graf besar sehingga pangkatnya menjadi 1,40. Penghentian dini mempercepat graf acak hingga 353 kali pada 4.000 simpul. Dijkstra 3,7 sampai 8,0 kali lebih cepat pada bobot nonnegatif, tetapi salah pada studi kasus yang berbobot negatif.***

***Kata kunci—Bellman-Ford, jalur terpendek, bobot negatif, siklus negatif, kompleksitas algoritma***

## I. PENDAHULUAN

### A. Latar Belakang

Banyak persoalan pada jaringan, misalnya mencari rute berbiaya terkecil dari satu titik ke titik-titik lain, dirumuskan sebagai masalah jarak terpendek pada graf berarah berbobot [1]. Algoritma Dijkstra menyelesaikan masalah ini secara efisien, tetapi benar hanya bila semua bobot sisi nonnegatif [1]. Pada sejumlah model, bobot sisi dapat bernilai negatif, yaitu ketika melewati suatu sisi justru mengurangi total biaya. Model seperti itu memerlukan algoritma yang tidak menetapkan jarak suatu simpul sebagai final terlalu dini.

Algoritma Bellman-Ford memenuhi kebutuhan tersebut. Algoritma ini memeriksa seluruh sisi berulang kali dan memperbarui estimasi jarak setiap kali ditemukan rute yang lebih murah. Metode ini diusulkan oleh Shimbel pada 1955 [2], dimuat dalam laporan Ford pada 1956 [3], dan diterbitkan oleh Bellman pada 1958 [4]. Moore menerbitkan varian serupa pada 1959 [5], sehingga algoritma ini kadang disebut Bellman-Ford-Moore. Selain menangani bobot negatif, algoritma ini mendeteksi siklus negatif yang terjangkau dari simpul sumber [1].

Gagasan Bellman-Ford dipakai dalam praktik jaringan. Versi terdistribusinya menjadi dasar protokol routing distance-vector seperti RIP, yang membatasi jarak sampai 15 lompatan [6]. Pada bidang keuangan, sebuah kerangka riset memodelkan kurs sebagai graf dan mendeteksi peluang arbitrase segitiga sebagai siklus negatif [7].

Selama puluhan tahun, batas waktu $O(|V| \cdot |E|)$ menjadi acuan untuk graf dengan bobot negatif. Dalam beberapa tahun terakhir batas itu mulai dilampaui. Bernstein dkk. memperoleh algoritma acak berwaktu hampir linear untuk bobot bulat [8]. Untuk bobot real, Fineman memperoleh waktu $\tilde{O}(mn^{8/9})$ [9] dan Huang dkk. memperbaikinya menjadi $\tilde{O}(mn^{4/5})$ [10]. Hasil-hasil itu diukur terhadap batas Bellman-Ford klasik, sehingga pemahaman yang rinci tentang algoritma klasik tetap menjadi dasar.

Tugas ini menganalisis algoritma Bellman-Ford klasik: menjelaskan prinsip kerjanya, menurunkan kebenaran dan kompleksitasnya, mengimplementasikannya dari nol, dan mengukur waktu eksekusinya terhadap kurva kompleksitas teoretis.

### B. Definisi Formal Masalah

Secara sederhana, masalahnya adalah mencari biaya termurah dari satu titik awal ke semua titik lain pada jaringan jalan satu arah, dengan biaya yang boleh bernilai negatif. Pada kasus kurir di Bagian III, titik adalah gudang dan toko, jalan adalah rute antartitik, dan biaya negatif adalah muatan balik yang menutup sebagian ongkos.

Dalam bahasa graf, jaringan itu ditulis $G = (V, E)$. $V$ adalah himpunan simpul (titik) dan $E$ adalah himpunan sisi (jalan satu arah). Setiap sisi $(u, v)$ punya bobot $w(u, v)$, yaitu biayanya, dan bobot ini boleh negatif. Simpul awal disebut sumber dan ditulis $s$ [1].

Jalur dari $s$ ke $v$ adalah urutan simpul $p = \langle v_0, v_1, \ldots, v_k \rangle$ yang dimulai di $s$ ($v_0 = s$) dan berakhir di $v$ ($v_k = v$). Setiap dua simpul yang berurutan dihubungkan oleh sebuah sisi. Bobot jalur $w(p)$ adalah jumlah bobot semua sisinya.

Jarak terpendek $\delta(s, v)$ adalah bobot jalur termurah dari $s$ ke $v$. Ada tiga kemungkinan:

- $\delta(s, v) = \infty$ bila tidak ada jalur dari $s$ ke $v$;
- $\delta(s, v) = -\infty$ bila jalur ke $v$ bisa melewati siklus negatif, yaitu putaran yang total bobotnya negatif. Setiap kali putaran itu diulang, biayanya turun lagi, sehingga tidak ada nilai yang paling murah [1];
- selain itu, $\delta(s, v) = \min\{ w(p) : p \text{ jalur dari } s \text{ ke } v \}$.

Masalah jarak terpendek satu sumber (single-source shortest paths, SSSP):

- **Masukan:** graf $G$, bobot $w$, dan sumber $s$.
- **Keluaran:** $\delta(s, v)$ untuk setiap simpul $v$, beserta jalur termurahnya bila jaraknya bukan $\infty$, atau pemberitahuan bahwa ada siklus negatif yang bisa dicapai dari $s$.

### C. Tujuan

(1) Menjelaskan cara kerja Bellman-Ford, membuktikan bahwa hasilnya selalu benar, dan menurunkan kompleksitas waktu dan memorinya, yaitu seberapa cepat kebutuhan waktu dan memori bertambah ketika graf makin besar.

(2) Membuat program Bellman-Ford sendiri, lalu mengujinya dengan contoh kecil yang bisa dihitung manual dan sebuah studi kasus distribusi barang, serta mencocokkan hasilnya dengan pustaka scipy.

(3) Mengukur lama program berjalan pada graf berbagai ukuran, lalu membandingkannya dengan perkiraan teori, dengan versi yang selalu menjalankan semua putaran tanpa berhenti lebih awal, serta dengan Dijkstra.

### D. Batasan

(1) Laporan ini hanya membahas pencarian jarak terpendek dari satu simpul awal ke semua simpul lain pada graf berarah yang bobotnya boleh negatif. Bila ada siklus negatif, program hanya memberi tahu bahwa siklus itu ada. Program tidak menunjukkan letak siklusnya dan tidak menandai simpul mana saja yang terpengaruh. Versi Bellman-Ford yang dijalankan di banyak prosesor, di banyak komputer sekaligus, atau di komputer kuantum [11] tidak dibuat.

(2) Program ditulis dalam Python dan dijalankan pada satu komputer, sehingga ukuran graf yang bisa diuji dibatasi oleh lama waktu jalan dan kapasitas memori. Graf terbesar yang diuji berisi 12.800 simpul untuk kasus terburuk dan 300.000 simpul untuk dua jenis graf lainnya. Alasannya ada di Bagian IV-D.

(3) Graf untuk eksperimen dibuat oleh program, bukan diambil dari data nyata, dan jumlah sisinya selalu empat kali jumlah simpul. Angka pada studi kasus distribusi dan kasus tukar uang juga rekaan.

(4) Algoritma lain yang secara teori lebih cepat daripada Bellman-Ford [8], [9], [10] hanya dibahas sebagai penelitian terkait dan tidak diimplementasikan.

## II. Landasan Teori

### A. Paradigma Desain

Bellman-Ford bekerja dengan memperbaiki perkiraan berulang kali. Untuk setiap simpul v, algoritma menyimpan perkiraan jarak d[v]. Awalnya perkiraan ini ∞, lalu diturunkan setiap kali ditemukan jalan yang lebih murah. Perkiraan itu tidak pernah lebih kecil daripada jarak sebenarnya. Karena itu, begitu sama dengan jarak sebenarnya, nilainya tidak berubah lagi [1]. Langkah memperbaiki perkiraan ini disebut relaksasi dan dijelaskan di Bagian II-B.

Dijkstra memakai strategi serakah (greedy). Setiap kali, Dijkstra mengambil simpul dengan perkiraan terkecil dan langsung menganggap jaraknya final. Cara ini aman bila semua bobot nonnegatif. Bila ada bobot negatif, cara ini bisa salah, karena jalan yang lebih murah bisa baru ditemukan belakangan. Bellman-Ford tidak pernah mengunci jarak lebih awal: semua sisi diperiksa ulang pada setiap putaran.

Cara kerja ini termasuk pemrograman dinamis (dynamic programming), yaitu jawaban besar dibangun dari jawaban kecil yang sudah dihitung sebelumnya [1], [4]. Dasarnya adalah sifat bahwa potongan awal dari jalur termurah juga merupakan jalur termurah. Misalkan $d_k(v)$ adalah biaya termurah ke v bila hanya boleh memakai paling banyak k sisi. Maka:

- $d_0(s) = 0$ dan $d_0(v) = \infty$ untuk $v \neq s$, karena tanpa sisi hanya sumber yang tercapai;
- $d_k(v) = \min( d_{k-1}(v), \min_{(u,v)\in E} [ d_{k-1}(u) + w(u, v) ] )$.

Rumus kedua berarti biaya termurah ke v dengan paling banyak k sisi adalah yang lebih kecil dari dua pilihan. Pilihan pertama adalah biaya termurah dengan paling banyak $k - 1$ sisi. Pilihan kedua adalah biaya termurah ke salah satu simpul u ditambah ongkos sisi dari u ke v. Satu putaran Bellman-Ford sama dengan menaikkan k satu langkah. Jalur termurah yang tidak mengulang simpul memuat paling banyak $|V| - 1$ sisi, jadi $|V| - 1$ putaran sudah cukup.

Program langsung memakai nilai d terbaru di tengah putaran, tanpa menunggu putaran selesai. Bagian III-C membuktikan bahwa cara ini tetap memberi hasil yang benar.

### B. Konsep Matematis

Untuk setiap simpul v, algoritma menyimpan dua nilai:

- d[v], yaitu perkiraan jarak dari s ke v sejauh ini;
- π[v] (pendahulu), yaitu simpul tepat sebelum v pada jalan termurah yang sejauh ini diketahui. Jalurnya bisa disusun kembali dengan mengikuti π mundur dari v sampai ke s.

Awalnya $d[s] = 0$, $d[v] = \infty$ untuk simpul lain, dan semua π[v] kosong.

Relaksasi sisi (u, v) berarti memeriksa apakah lewat u lebih murah, yaitu apakah $d[u] + w(u, v) < d[v]$. Bila ya, d[v] diganti menjadi $d[u] + w(u, v)$ dan π[v] diganti menjadi u [1]. Contohnya pada kasus kurir: d[Toko B] = 5 dan sisi Toko B → Toko A berbobot −3, sehingga lewat Toko B biayanya $5 + (-3) = 2$. Nilai ini lebih kecil daripada d[Toko A] = 4, jadi d[Toko A] menjadi 2 dan π[Toko A] menjadi Toko B.

Tiga sifat berikut dipakai pada bukti di Bagian III [1].

(a) Perkiraan tidak pernah terlalu kecil. Sepanjang algoritma $d[v] \geq \delta(s, v)$. Karena d hanya bisa turun, begitu $d[v] = \delta(s, v)$ nilainya tidak berubah lagi.

(b) Simpul yang tidak bisa dicapai tetap ∞. Bila tidak ada jalur dari s ke v, maka $d[v] = \delta(s, v) = \infty$ sampai akhir.

(c) Jalur termurah cukup memakai paling banyak $|V| - 1$ sisi. Bila tidak ada siklus negatif yang bisa dicapai dari s, setiap simpul yang bisa dicapai punya jalur termurah yang tidak mengulang simpul. Jalur seperti itu melewati paling banyak |V| simpul, jadi paling banyak $|V| - 1$ sisi.

### C. Penelitian Terkait

Penelitian tentang Bellman-Ford sejak 1958 bergerak ke dua arah: mengurangi jumlah pemeriksaan sisi, dan mencari algoritma yang lebih cepat daripada batas $O(|V|\cdot|E|)$.

Arah pertama mengatur urutan pemeriksaan sisi. Yen mengatur urutan itu sehingga jumlah relaksasi turun sekitar separuh [12], [13]. Bannister dan Eppstein mengacak urutan simpul pada setiap putaran. Dengan cara itu, jumlah relaksasi pada kasus terburuk turun lagi menjadi sekitar dua pertiga dari varian Yen, dengan peluang yang sangat besar. Mereka juga menambahkan deteksi siklus negatif pada varian acak ini [14]. Hu dan Kozma, dalam sebuah preprint, menunjukkan bahwa

penghematan separuh dari Yen sudah yang terbaik untuk metode yang urutannya ditetapkan dari awal dan tanpa unsur acak [13].

Arah kedua mencari algoritma yang lebih cepat secara mendasar. Pada notasi berikut, n = |V|, m = |E|, dan tanda $\tilde{O}$ berarti faktor logaritma diabaikan. Untuk bobot bilangan bulat, Bernstein dkk. memperoleh algoritma acak berwaktu $O(m \log^8 n \cdot \log W)$, dengan $-W$ bobot paling negatif. Waktu ini hampir sebanding dengan jumlah sisi saja, dan makalahnya menjadi salah satu Best Paper FOCS 2022 [8]. Untuk bobot bilangan real, Fineman memperoleh $\tilde{O}(mn^{8/9})$, perbaikan pertama atas O(mn) setelah puluhan tahun [9]. Huang dkk. memperbaikinya lagi menjadi $\tilde{O}(mn^{4/5})$ [10].

Ashvinkumar dkk. membawa hasil Bernstein dkk. ke komputer paralel, jaringan terdistribusi, dan komputer kuantum. Caranya adalah mengubah masalah berbobot negatif menjadi beberapa kali pemanggilan algoritma jarak terpendek yang lebih sederhana [11]. Dari sisi penerapan, Akouaouch dan Bouayad memakai Bellman-Ford dalam kerangka waktu nyata untuk mendeteksi arbitrase segitiga di pasar kripto [7].

### *D. Posisi Tugas Ini*

Penelitian di atas bergerak ke arah mempercepat Bellman-Ford atau memperluasnya ke model lain. Tugas ini mengambil arah yang berbeda dan tidak mengajukan algoritma baru. Fokusnya adalah membedah Bellman-Ford klasik: menurunkan kebenaran dan kompleksitasnya, mengimplementasikannya dari nol, dan membandingkan hasil ukur dengan kurva teoretis.

## III. Analisis Algoritma

### *A. Deskripsi dan Pseudocode*

Bellman-Ford menyelesaikan masalah pada Bagian I-B dengan relaksasi berulang. Untuk setiap simpul v, algoritma menyimpan perkiraan jarak d[v] dan pendahulu π[v] (Bagian II-B). Satu putaran berarti merelaksasi setiap sisi satu kali, sesuai urutan sisi pada masukan. Putaran diulang |V| − 1 kali, karena jalur termurah yang tidak mengulang simpul memuat paling banyak |V| − 1 sisi (sifat (c), Bagian II-B). Setelah itu ada satu putaran tambahan untuk memeriksa apakah ada siklus negatif yang terjangkau dari s, yaitu yang bisa dicapai dari s. Pseudocode berikut mengikuti [1], dan nomor barisnya dipakai lagi pada kode di Bagian IV.

**Algoritma 1.** BELLMAN-FORD(G, w, s)

```
1  untuk setiap v ∈ V:  d[v] ← ∞;  π[v] ← NIL
2  d[s] ← 0
3  ulangi |V| − 1 kali:
4      untuk setiap sisi (u, v) ∈ E:
5          jika d[u] < ∞ dan d[u] + w(u, v) < d[v]:
6              d[v] ← d[u] + w(u, v);  π[v] ← u
7  untuk setiap sisi (u, v) ∈ E:
8      jika d[u] < ∞ dan d[u] + w(u, v) < d[v]:
           kembalikan (d, π, BENAR)
9  kembalikan (d, π, SALAH)
```

Cara membaca pseudocode:

- **Baris 1–2:** nilai awal. Semua simpul bernilai ∞, kecuali sumber bernilai 0.
- **Baris 3–6:** inti algoritma. Setiap putaran (baris 3) memeriksa semua sisi (baris 4). Bila lewat u lebih murah untuk sampai ke v (baris 5), d[v] dan π[v] diperbarui (baris 6).
- **Baris 7–8:** pemeriksaan sekali lagi. Bila masih ada sisi yang bisa membuat jarak lebih murah, berarti ada siklus negatif dan nilai d tidak bisa dipakai.
- **Baris 9:** tidak ada siklus negatif, jadi d dan π adalah jawabannya.

Dua detail pada baris 5 penting untuk kebenaran. Pertama, syarat $d[u] < \infty$ mencegah sisi dari simpul yang belum pernah dicapai ikut dipakai. Tanpa syarat ini, jarak palsu bisa muncul bila ∞ ditulis sebagai bilangan besar biasa, misalnya $10^9$. Kedua, perbandingannya memakai < dan bukan ≤, sehingga biaya yang sama tidak dianggap perbaikan dan π tidak berganti tanpa alasan.

Program di Bagian IV memuat satu tambahan: perulangan berhenti lebih awal begitu satu putaran tidak mengubah d sama sekali. Tambahan ini tidak mengubah hasil (Bagian III-C1) dan hanya bisa mengurangi jumlah putaran. Pseudocode di atas adalah versi baku tanpa tambahan itu. Kedua versi diukur terpisah di Bagian IV-E.

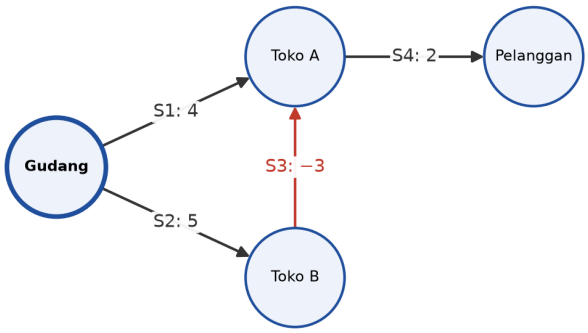
Jalur termurah ke v disusun dengan mengikuti π mundur dari v sampai s. Penelusuran berhenti saat tiba di s, bukan saat menemukan π kosong. Alasannya, s dan simpul yang tidak bisa dicapai sama-sama punya π kosong, jadi π kosong saja tidak cukup untuk membedakan keduanya.

### *B. Contoh Perhitungan Manual*

Contoh dihitung pada graf kurir empat simpul pada Gambar 1. Kurir berangkat dari Gudang menuju Pelanggan. Ia bisa lewat Toko A, atau mampir dulu ke Toko B. Ruas Toko B → Toko A membawa titipan berbayar, sehingga biaya bersihnya −3. Sisi diperiksa dengan urutan berikut:

- S1: Gudang → Toko A (4)
- S2: Gudang → Toko B (5)
- S3: Toko B → Toko A (−3)
- S4: Toko A → Pelanggan (2)

Nilai yang baru diperbarui langsung dipakai oleh sisi berikutnya dalam putaran yang sama.




Gambar 1. Graf kasus kurir. Angka pada panah adalah bobot sisi, merah untuk bobot negatif.

*Putaran 1.*

- S1: 0 + 4 = 4, lebih murah dari ∞, jadi d[Toko A] = 4.
- S2: 0 + 5 = 5, jadi d[Toko B] = 5.
- S3: 5 + (−3) = 2, lebih murah dari 4, jadi d[Toko A] turun menjadi 2.

- S4: langsung memakai nilai baru tadi, 2 + 2 = 4, jadi d[Pelanggan] = 4.

*Putaran 2.* Keempat sisi diperiksa lagi, tetapi tidak ada yang lebih murah. S1 memberi 4, padahal d[Toko A] sudah 2. S2 memberi 5, sama dengan d[Toko B]. S3 memberi 2, sama dengan d[Toko A]. S4 memberi 4, sama dengan d[Pelanggan]. Karena tidak ada yang berubah, perulangan berhenti. Pemeriksaan siklus negatif (baris 7–8) juga tidak menemukan sisi yang bisa diperbarui. Tabel I merangkum nilai d pada tiap tahap.

TABEL I. NILAI D SETELAH TIAP PUTARAN PADA GRAF GAMBAR 1

| Putaran | Gudang | Toko A | Toko B | Pelanggan |
|---|---|---|---|---|
| Awal | 0 | ∞ | ∞ | ∞ |
| 1 | 0 | 2 | 5 | 4 |
| 2 | 0 | 2 | 5 | 4 |

Hasil akhirnya d = (0, 2, 5, 4), dengan π[Toko B] = Gudang, π[Toko A] = Toko B, dan π[Pelanggan] = Toko A. Bila π diikuti mundur dari Pelanggan, jalurnya adalah Gudang → Toko B → Toko A → Pelanggan, dengan biaya 5 − 3 + 2 = 4.

Jalur ke Pelanggan memuat 3 sisi, sama dengan $|V| - 1$. Meski begitu, semua nilai sudah benar dalam satu putaran, karena urutan S2, S3, S4 kebetulan searah dengan jalur itu. Bila urutannya dibalik, putaran yang diperlukan lebih banyak. Kasus 3 di Bagian IV-C menunjukkan hal ini. Dijkstra pada graf yang sama memberi Toko A = 4 dan Pelanggan = 6 (Bagian III-E). Hasil program sama dengan hitungan ini (Bagian IV-C).

### C. Bukti Kebenaran

Ada dua hal yang perlu dibuktikan: jaraknya benar setelah $|V| - 1$ putaran, dan siklus negatif selalu terdeteksi.

#### 1) Jarak Benar Setelah $|V| - 1$ Putaran

**Klaim 1.** Bila tidak ada siklus negatif yang terjangkau dari s, maka setelah putaran ke-k, setiap simpul yang jalur termurahnya berisi paling banyak k sisi sudah bernilai benar, yaitu $d[v] = \delta(s, v)$.

*Bukti* (induksi pada k). Untuk k = 0 hanya ada s, dan d[s] = 0 sudah benar sejak awal. Untuk $k \geq 1$, ambil v dengan jalur termurah $s \to \dots \to u \to v$ yang berisi paling banyak k sisi. Potongan $s \to \dots \to u$ berisi paling banyak $k - 1$ sisi, jadi d[u] sudah benar setelah putaran $k - 1$. Pada putaran k, sisi (u, v) diperiksa, sehingga d[v] turun paling sedikit sampai $d[u] + w(u, v) = \delta(s, v)$. Karena d[v] tidak pernah di bawah $\delta(s, v)$ (sifat (a)), maka $d[v] = \delta(s, v)$. ■

**Akibat 1 ($|V| - 1$ putaran cukup).** Jalur termurah berisi paling banyak $|V| - 1$ sisi (sifat (c)), jadi setelah $|V| - 1$ putaran semua jarak benar. Ini berlaku untuk urutan sisi apa pun. Urutan hanya menentukan seberapa cepat nilai benar tercapai: contoh kurir selesai dalam 1 putaran, sedangkan rantai terbalik memerlukan 3 putaran.

**Akibat 2 (penghentian dini aman).** Bila satu putaran tidak mengubah apa pun, putaran berikutnya memeriksa sisi yang sama dengan nilai d yang sama, sehingga juga tidak mengubah apa pun. Jadi berhenti di situ memberi hasil yang sama dengan menjalankan semua putaran, termasuk hasil uji siklus negatif.

#### 2) Siklus Negatif Selalu Terdeteksi

**Klaim 2.** Uji pada baris 7–8 melaporkan siklus negatif jika dan hanya jika ada siklus negatif yang terjangkau dari s.

*Bukti.* Bila tidak ada siklus negatif, semua d sudah benar menurut Akibat 1. Jalan termurah ke v tidak mungkin lebih mahal daripada jalan lewat u, jadi $d[v] \leq d[u] + w(u, v)$ untuk setiap sisi, dan tidak ada sisi yang lolos uji.

Bila ada siklus negatif $c = \langle v_0, v_1, \dots, v_k = v_0 \rangle$ yang terjangkau dari s, andaikan tidak ada sisi yang lolos uji. Maka $d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i)$ untuk setiap sisi pada siklus. Semua nilai d ini terhingga, karena siklusnya terjangkau dari s. Bila semua ketaksamaan itu dijumlahkan, jumlah d di kiri dan kanan sama, karena siklus kembali ke titik awalnya. Yang tersisa adalah $0 \leq w(c)$, padahal $w(c) < 0$. Jadi pasti ada sisi yang lolos uji. ■

Contohnya segitiga tukar uang dengan bobot keliling −36. Setelah putaran 2, sisi USD → EUR masih lolos uji ($-72 + 105 = 33 < 69$), sehingga siklus dilaporkan (Lampiran C). Siklus negatif yang tidak terjangkau dari s tidak terdeteksi, dan ini sesuai dengan Bagian I-B.

### D. Analisis Kompleksitas

Inti analisisnya sederhana: waktu Bellman-Ford sama dengan banyaknya putaran dikali banyaknya sisi. Setiap putaran memeriksa semua sisi, dan setiap pemeriksaan memakan waktu tetap. Jadi yang perlu dicari hanya berapa putaran yang dijalankan.

Misalkan $n = |V|$, $m = |E|$, dan r adalah banyaknya putaran. Dengan tambahan pengisian nilai awal (n) dan uji siklus negatif (m), banyaknya langkah adalah

$$T = n + r \cdot m + m.$$

Versi baku selalu menjalankan $r = n - 1$ putaran, sehingga $T = \Theta(n \cdot m)$ untuk semua masukan. Versi dengan penghentian dini berhenti begitu satu putaran tidak mengubah apa pun, sehingga r berada di antara 1 dan $n - 1$ (Akibat 1). Notasi $\Theta(f)$ berarti waktu tumbuh sebanding dengan f.

- **Kasus terbaik:** urutan sisi searah dengan jalur termurah. Putaran pertama langsung benar dan putaran kedua hanya memastikan, jadi $r = 2$ dan $T = \Theta(n + m)$. Tidak ada yang lebih cepat dari ini, karena setiap simpul harus diberi nilai awal dan setiap sisi harus dibaca minimal sekali.
- **Kasus terburuk:** urutan sisi berlawanan arah dengan jalur termurah, atau ada siklus negatif. Setiap putaran hanya memperbaiki satu simpul, jadi $r = n - 1$ dan $T = \Theta(n \cdot m)$. Tidak ada yang lebih lambat dari ini, karena r tidak pernah lebih dari $n - 1$.
- **Kasus rata-rata:** r paling banyak sama dengan jumlah sisi pada jalur termurah yang terpanjang, ditambah satu (Klaim 1). Pada graf acak, jalur itu diduga hanya sepanjang sekitar $\log n$, sehingga T mendekati $\Theta(m \log n)$. Dugaan ini diuji di Bagian IV.

Contohnya rantai empat simpul pada kasus 3 ($n = 4$, $m = 3$). Daftar searah selesai dalam $r = 2$ putaran, dengan $2 \cdot 3 + 3 = 9$ pemeriksaan sisi. Daftar terbalik memerlukan $r = 3$, dengan $3 \cdot 3 + 3 = 12$ pemeriksaan sisi.

Untuk ruang, algoritma hanya menyimpan dua larik sepanjang n, yaitu d dan π. Jadi memori tambahannya $\Theta(n)$. Graf disimpan sebagai daftar sisi. Bila graf disimpan sebagai

matriks n × n, setiap putaran harus memeriksa $n^2$ pasangan simpul, dan totalnya menjadi $\Theta(n^3)$.

Tabel II merangkum kompleksitas waktu menurut versi dan kasus.

TABEL II. RINGKASAN KOMPLEKSITAS WAKTU MENURUT VERSI DAN KASUS

| Kasus | Putaran r | Waktu |
|---|---|---|
| Versi baku, semua kasus | n − 1 | $\Theta(n \cdot m)$ |
| Henti dini, terbaik | 1–2 | $\Theta(n + m)$ |
| Henti dini, rata-rata (graf acak) | sekitar log n | $\approx \Theta(m \log n)$ |
| Henti dini, terburuk | n − 1 | $\Theta(n \cdot m)$ |

### E. *Perbandingan dengan Algoritma Lain*

Pembanding utama adalah algoritma Dijkstra dengan antrean prioritas berupa heap biner. Dijkstra selalu mengambil simpul dengan d terkecil yang belum final, langsung menganggap jaraknya final, lalu memeriksa sisi keluarnya. Cara ini benar bila semua bobot nonnegatif. Dalam keadaan itu, menambah sisi tidak pernah mengurangi biaya, jadi simpul yang sudah final tidak mungkin dicapai lebih murah lewat jalan lain [1].

Dengan bobot negatif, anggapan itu tidak berlaku. Pada kasus kurir, Dijkstra memfinalkan Toko A = 4 lebih dulu, karena 4 lebih kecil daripada Toko B = 5. Setelah itu baru sisi Toko B → Toko A (−3) diperiksa, dan sisi ini memberi 5 − 3 = 2. Namun Toko A sudah dikunci. Akibatnya Toko A tetap 4 dan Pelanggan 6, padahal jawaban yang benar 2 dan 4. Studi kasus di Bagian IV-C menunjukkan kegagalan yang sama pada skala lebih besar. Bellman-Ford tidak mengunci apa pun sebelum selesai, sehingga bisa menerima bobot negatif dan mendeteksi siklus negatif. Harganya adalah waktu yang lebih lama.

TABEL III. PERBANDINGAN BELLMAN-FORD DENGAN DIJKSTRA

| Kriteria | Bellman-Ford | Dijkstra (heap biner) |
|---|---|---|
| Bobot sisi yang diterima | real, termasuk negatif | nonnegatif |
| Siklus negatif | terdeteksi bila terjangkau dari s | tidak terdeteksi; hasil salah bila ada bobot negatif |
| Paradigma | relaksasi berulang pada semua sisi | serakah, simpul difinalkan satu per satu |
| Waktu terburuk | $\Theta(\|V\| \cdot \|E\|)$ | $O((\|V\| + \|E\|) \log \|V\|)$ [1] |
| Struktur data | daftar sisi dan dua larik (d, π) | daftar tetangga, antrean prioritas, larik d dan π |
| Cocok untuk | bobot negatif, deteksi siklus negatif (misalnya arbitrase), routing distance-vector | bobot nonnegatif dan graf besar |

Pada graf jarang, yaitu bila jumlah sisi sebanding dengan jumlah simpul, Dijkstra berbiaya $O(|V| \log |V|)$, sedangkan Bellman-Ford pada kasus terburuk $O(|V|^2)$. Kedua algoritma juga bisa digabung. Algoritma Johnson menjalankan Bellman-Ford sekali untuk mengubah semua bobot menjadi nonnegatif tanpa mengubah jalur termurahnya, lalu menjalankan Dijkstra dari setiap simpul [1]. Eksperimen di Bagian IV juga memakai Bellman-Ford dari pustaka scipy sebagai acuan kebenaran.

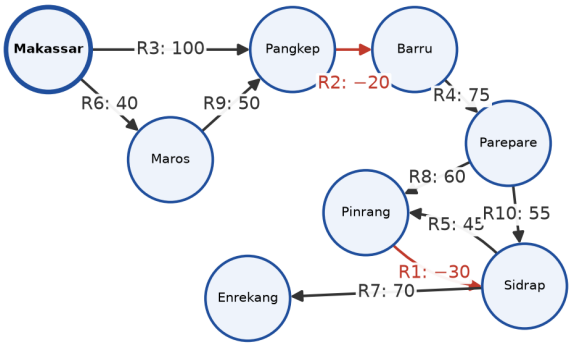
## IV. IMPLEMENTASI DAN EKSPERIMEN

### A. *Studi Kasus dan Lingkungan Uji*

Studi kasusnya rekaan. Sebuah ekspedisi mengirim barang dari Makassar ke tujuh kota di Sulawesi Selatan, dan manajernya ingin tahu biaya bersih termurah ke setiap kota. Biaya bersih sebuah ruas adalah biaya operasional dikurangi pendapatan dari muatan balik. Pada dua ruas, pendapatan muatan lebih besar daripada biayanya, sehingga biaya bersihnya negatif:

- R1 (Pinrang → Sidrap): 50 − 80 = −30
- R2 (Pangkep → Barru): 60 − 80 = −20

Satuannya ribu rupiah per ton. Semua angka dibuat sendiri untuk tugas ini, sedangkan nama kota hanya memberi konteks. Gambar 2 menunjukkan jaringannya. Studi kasus ini menguji jarak dengan bobot negatif pada graf yang lebih besar daripada kasus kecil, dan dipakai untuk membandingkan Bellman-Ford dengan Dijkstra serta melihat pengaruh urutan sisi (Bagian IV-C).




Gambar 2. Jaringan studi kasus distribusi barang. Angka pada panah adalah biaya bersih ruas, merah untuk bobot negatif.

Siklus negatif sudah ditunjukkan oleh kasus segitiga tukar uang, dan simpul yang tidak terjangkau diperiksa lewat uji silang (Bagian IV-C), jadi keduanya tidak diberi skenario sendiri. Simpul tanpa jalur dari sumber tetap ∞, karena algoritma tidak pernah memakai sisi dari simpul yang masih ∞ (sifat (b)).

Lingkungan uji ada pada Tabel IV. Prosesor, sistem operasi, serta versi Python dan numpy dicetak langsung oleh sel pertama notebook eksperimen.

TABEL IV. SPESIFIKASI LINGKUNGAN UJI

| Komponen | Spesifikasi |
|---|---|
| Prosesor | Apple M2 Pro (10 inti) |
| Memori | 16 GB |
| Sistem operasi | macOS 27.0.1 (arm64) |
| Bahasa pemrograman | Python 3.14.7 |
| Pustaka | numpy 2.5.3, matplotlib 3.11.2, scipy 1.18.1 (kunci jawaban uji silang) |
| Pengukuran waktu | time.perf_counter, satu pemanggilan algoritma |

### B. *Implementasi*

Program terdiri atas dua notebook (Lampiran A):

- **Bellman_Ford_Sederhana.ipynb** berisi inti algoritma, tiga kasus kecil, dan studi kasus. Notebook ini tidak memakai pustaka apa pun.
- **Bellman_Ford_Eksperimen.ipynb** berisi inti yang sama, uji silang dengan scipy, dan eksperimen waktu. Pustaka numpy, matplotlib, dan scipy hanya dipakai di luar inti, yaitu untuk statistik, grafik, dan pencocokan hasil.

Graf disimpan sebagai daftar sisi (u, v, w), sedangkan d dan π disimpan sebagai dua list Python sepanjang |V|. Daftar sisi dipilih karena setiap putaran hanya perlu membaca semua sisi satu kali (Bagian III-D). Nilai ∞ ditulis `float('inf')`. Listing 1 menampilkan inti kode. Nomor baris di komentarnya mengacu ke Algoritma 1, dan Lampiran B memetakan setiap baris pseudocode ke kodenya.

**Listing 1.** Inti implementasi Bellman-Ford (Python), nomor baris mengacu ke Algoritma 1

```
INF = float("inf")

def relaksasi(u, v, w, dist, pred):      # baris 5-6
    if dist[u] != INF and dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        pred[v] = u
        return True
    return False

def satu_putaran(edges, dist, pred):     # baris 4
    ada_perubahan = False
    for u, v, w in edges:
        if relaksasi(u, v, w, dist, pred):
            ada_perubahan = True
    return ada_perubahan

def ada_siklus_negatif(edges, dist):     # baris 7-8
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            return True
    return False

def bellman_ford(n, edges, src, berhenti_dini=True):
    dist, pred = [INF] * n, [None] * n   # baris 1
    dist[src] = 0                        # baris 2
    for _ in range(n - 1):               # baris 3
        ada_perubahan = satu_putaran(edges, dist, pred)
        if berhenti_dini and not ada_perubahan:
            break                        # henti dini
    return dist, pred, ada_siklus_negatif(edges, dist)
```

Kode mengikuti pseudocode baris demi baris, termasuk dua detail pada baris 5 yang dibahas di Bagian III-A. Ada tiga hal yang khusus pada implementasi:

- `dist` dan `pred` diubah langsung di tempat. Jadi nilai baru di tengah putaran langsung dipakai oleh sisi berikutnya, seperti pada hitungan di Bagian III-B.
- Parameter `berhenti_dini` menghentikan perulangan bila satu putaran tidak mengubah apa pun (Akibat 2). Bila diisi `False`, program menjalankan versi baku. Versi baku inilah yang dibandingkan di Bagian IV-E.
- Fungsi `rekonstruksi_jalur(pred, src, target)` menyusun jalur dengan mengikuti pred mundur sampai src. Fungsi ini juga membatasi jumlah langkah, karena bila ada siklus negatif, pred bisa membentuk lingkaran dan penelusuran tidak akan pernah berhenti.

### C. Pengujian Kebenaran

Program diuji dengan tiga cara: kasus kecil yang dihitung tangan, studi kasus, dan uji silang dengan scipy. Tabel V merangkum hasil kasus kecil dan studi kasus. Harapan setiap kasus dihitung tangan putaran demi putaran (kasus 1 di Bagian III-B, kasus lain di Lampiran C), lalu dicocokkan dengan assert di notebook. Semua kasus lulus.

TABEL V. KASUS UJI DAN HASILNYA

| No. | Keterangan | Hasil hitung tangan | Keluaran program | Sama? |
|---|---|---|---|---|
| 1 | Rute kurir 4 titik, satu bobot negatif (Gambar 1) | d = (0, 2, 5, 4) | d = (0, 2, 5, 4) | Ya |
| 2 | Segitiga tukar uang, keliling −36 | siklus negatif terdeteksi | siklus negatif terdeteksi | Ya |
| 3 | Rantai 4 titik, daftar terbalik dan searah | d = (0, 2, 5, 6); 3 dan 2 putaran | d = (0, 2, 5, 6); 3 dan 2 putaran | Ya |
| 4 | Studi kasus distribusi (Gambar 2) | d = (0, 40, 90, 70, 145, 175, 205, 245) | d = (0, 40, 90, 70, 145, 175, 205, 245) | Ya |

#### 1) Kasus Kecil

- **Kasus 1 (kurir)** menunjukkan bahwa bobot negatif ditangani dengan benar. Hitungannya ada di Bagian III-B.
- **Kasus 2 (segitiga tukar uang)** memakai tiga kurs rekaan: 1 USD = 0,9 EUR, 1 EUR = 160 JPY, dan 1 JPY = 0,0072 USD. Hasil kalinya 1,0368, artinya uang bertambah 3,68% setiap kali berkeliling. Bobot ditulis sebagai $-1000 \cdot \ln(\text{kurs})$ yang dibulatkan (105, −5075, dan 4934), sehingga keuntungan itu menjadi siklus berbobot −36 dan terdeteksi.
- **Kasus 3 (rantai)** menunjukkan pengaruh urutan sisi. Dengan daftar terbalik (C → D, B → C, A → B), setiap putaran hanya mengisi satu simpul, jadi ketiga putaran terpakai dan ada 12 pemeriksaan sisi. Dengan daftar searah, cukup 1 putaran ditambah 1 putaran konfirmasi, jadi 9 pemeriksaan. Jaraknya sama.

#### 2) Studi Kasus

Studi kasus memperlihatkan dua hal dari Bagian III:

- **Dijkstra klasik salah (Tabel VI).** Dijkstra mengunci Sidrap = 200 lewat Parepare, sebelum ruas Pinrang → Sidrap (−30) sempat dipakai. Jawaban yang benar adalah 175, lewat Parepare → Pinrang → Sidrap. Kesalahan ini menular ke Enrekang, yang bernilai 270 padahal seharusnya 245.
- **Urutan sisi hanya mengubah jumlah putaran (Tabel VII).** Urutan asli R1–R10 memerlukan 3 putaran berisi perubahan. Urutan dari Makassar ke luar hanya memerlukan 1. Urutan yang membalik jalur ke Enrekang memerlukan 7, yaitu $|V| - 1$. Hasil akhirnya sama (Akibat 1).

TABEL VI. BELLMAN-FORD DAN DIJKSTRA KLASIK PADA STUDI KASUS

| Kota | Bellman-Ford | Dijkstra klasik | Sama? |
|---|---|---|---|
| Makassar | 0 | 0 | ya |
| Maros | 40 | 40 | ya |
| Pangkep | 90 | 90 | ya |
| Barru | 70 | 70 | ya |
| Parepare | 145 | 145 | ya |
| Sidrap | 175 | 200 | tidak |
| Pinrang | 205 | 205 | ya |
| Enrekang | 245 | 270 | tidak |

TABEL VII. PENGARUH URUTAN SISI PADA STUDI KASUS (|V| − 1 = 7)

| Urutan | Ruas diperiksa | Putaran berisi perubahan | Hasil sama? |
|---|---|---|---|
| asli R1..R10 | R1, R2, R3, R4, R5, R6, R7, R8, R9, R10 | 3 | ya |
| dari Makassar ke luar | R3, R6, R9, R2, R4, R8, R10, R5, R1, R7 | 1 | ya |
| jalur Enrekang terbalik | R7, R1, R8, R4, R2, R9, R6, R3, R5, R10 | 7 | ya |

*3) Uji Silang dengan scipy*

Sebanyak 1.000 graf acak kecil (2–9 simpul, bobot −3 sampai 9, seed 2026) dijalankan pada program dan pada scipy di notebook eksperimen. Sebanyak 105 graf memuat siklus negatif dan 895 tidak. Bila generator yang sama dijalankan ulang, 715 dari 895 graf itu ternyata memiliki simpul yang tidak terjangkau. Hasil program dan scipy sama pada semua graf, baik deteksi siklus maupun seluruh jarak, termasuk nilai ∞ (0 selisih).

## D. Desain Eksperimen

Eksperimen memperbesar kasus-kasus di Bagian IV-C untuk menjawab dua pertanyaan:

(1) Apakah waktu eksekusi mengikuti perkiraan teori pada ketiga kasus di Bagian III-D?

(2) Seberapa besar pengaruh penghentian dini, dan seberapa mahal Bellman-Ford dibandingkan Dijkstra?

Keputusan desain dirangkum di Tabel VIII. Yang diukur hanya lama satu pemanggilan algoritma, tanpa waktu membangun graf atau daftar tetangga. scipy hanya dipakai sebagai kunci jawaban pada uji silang (Bagian IV-C), bukan sebagai pembanding waktu. Implementasinya dikompilasi dan selalu menjalankan semua putaran, sehingga tidak setara dengan program Python ini.

TABEL VIII. KEPUTUSAN DESAIN EKSPERIMEN

| Keputusan | Pilihan | Alasan |
|---|---|---|
| Keluarga graf | rantai searah (terbaik), graf acak (rata-rata), rantai terbalik (terburuk); \|E\| = 4\|V\| | mewakili tiga kasus Bagian III-D; \|E\| sebanding \|V\| membuat sumbu x tunggal (kurva teori c·\|V\| atau c·\|V\|²) |
| Ukuran | terburuk: 400 sampai 12.800; terbaik dan acak: 1.000 sampai 300.000; enam ukuran tiap keluarga | 12.800 pada kasus terburuk sudah sekitar 655 juta pemeriksaan sisi; 300.000 simpul (1,2 juta sisi) mendekati batas memori dan waktu pembangkitan graf di Python |
| Versi dan pembanding | dengan henti dini (utama); tanpa henti dini (\|V\| 500 sampai 4.000); Dijkstra heapq (bobot nonnegatif) | memisahkan pengaruh henti dini; Dijkstra menunjukkan harga dari kemampuan menerima bobot negatif |
| Pengulangan, pengukuran, seed | 5 kali, graf berbeda tiap ulangan (seed 2026 + r); time.perf_counter; jumlah putaran dicatat | simpangan baku memuat variasi graf; jumlah putaran menjelaskan perbedaan waktu antarkeluarga |

Ada tiga keluarga graf, dan masing-masing adalah kasus di Bagian IV-C yang diperbesar:

- **Terburuk:** kasus 3 dengan daftar terbalik. Rantai 0 → 1 → … → n − 1 berbobot acak 1–10, dan sisinya dimasukkan dari ujung ke pangkal, sehingga setiap putaran hanya memajukan satu simpul.
- **Terbaik:** kasus 3 dengan daftar searah. Rantainya sama, tetapi sisinya dimasukkan dari pangkal ke ujung, sehingga satu putaran sudah cukup, ditambah satu putaran konfirmasi.
- **Rata-rata:** mirip studi kasus distribusi, dengan bobot positif dan negatif tanpa siklus negatif, tetapi berupa graf acak yang jauh lebih besar. Graf dibangun dari pohon acak yang berakar di sumber: setiap simpul i memilih induk acak di antara simpul 0 sampai i − 1, sehingga semua simpul terjangkau. Setelah itu ditambah sisi acak, dan urutan sisinya diacak.

Ada dua pembanding waktu:

- **Versi tanpa henti dini**, yaitu Algoritma 1 apa adanya. Versi ini memperlihatkan seberapa besar penghematan dari berhenti lebih awal, seperti pada putaran 2 kasus kurir.
- **Dijkstra dengan heap biner**, dijalankan pada graf acak dengan bobot dasar yang nonnegatif supaya Dijkstra memberi hasil yang benar.

Supaya semua keluarga punya |E| = 4|V|, kedua rantai ditambah 3|V| sisi pengisi. Setiap sisi pengisi melompati paling sedikit dua simpul dan berbobot 10(v − u) + 1. Bobot ini lebih mahal daripada jalan lewat rantai, yang paling mahal 10(v − u). Jadi sisi pengisi tidak pernah menjadi bagian dari jalur termurah dan tidak mengurangi jumlah putaran. Fungsinya hanya menambah beban pemeriksaan.

Graf acak perlu bobot negatif tetapi tidak boleh punya siklus negatif. Caranya, bobot dasar bulat 1–100 digeser menjadi w′(u, v) = w(u, v) + p(u) − p(v), dengan p acak 0–50 untuk setiap simpul. Pada sembarang siklus, setiap p muncul sekali sebagai tambahan dan sekali sebagai pengurang. Jadi total bobot siklus tidak berubah dan tetap positif, sementara sebagian sisi menjadi negatif. Cara ini juga dipakai algoritma Johnson [1].

Pada setiap ukuran, semua algoritma dijalankan pada graf yang sama dan seluruh jaraknya dibandingkan. Bila ada yang berbeda, eksperimen berhenti. Dengan begitu setiap pengukuran sekaligus menjadi uji kebenaran.

## E. Hasil Eksperimen

Semua waktu disajikan dalam detik, sebagai rata-rata ± simpangan baku dari lima ulangan. Kolom putaran r adalah jumlah putaran yang dijalankan versi dengan henti dini, dirata-ratakan pada lima graf.

- **Tabel IX** memuat kasus terburuk. Jumlah putarannya selalu |V| − 1, dan waktunya naik dari 0,05 detik pada 400 simpul menjadi 40 detik pada 12.800 simpul.
- **Tabel X** memuat kasus terbaik dan rata-rata. Kasus terbaik selalu selesai dalam 2 putaran, sedangkan graf acak memerlukan 8,2 sampai 15,8 putaran.

TABEL IX. WAKTU EKSEKUSI PADA KASUS TERBURUK (RANTAI TERBALIK), DETIK

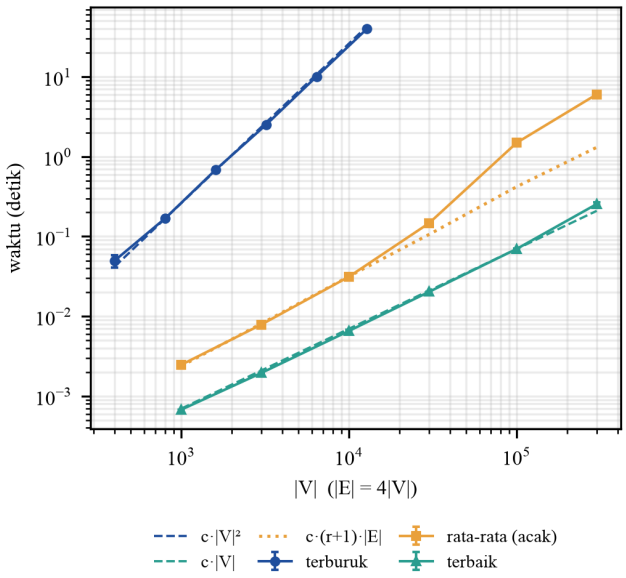
| \|V\| | \|E\| | Putaran r | Waktu (s) |
|---|---|---|---|
| 400 | 1.600 | 399 | 0,0498 ± 0,00874 |
| 800 | 3.200 | 799 | 0,168 ± 0,00716 |
| 1.600 | 6.400 | 1.599 | 0,684 ± 0,0358 |
| 3.200 | 12.800 | 3.199 | 2,51 ± 0,0399 |
| 6.400 | 25.600 | 6.399 | 10,1 ± 0,124 |
| 12.800 | 51.200 | 12.799 | 40,0 ± 0,506 |

TABEL X. WAKTU EKSEKUSI PADA KASUS TERBAIK DAN RATA-RATA, DETIK

| \|V\| | r (acak) | Terbaik (s) | Acak (s) |
|---|---|---|---|
| 1.000 | 8,2 | 0,000678 ± 0,0000020 | 0,00247 ± 0,000240 |
| 3.000 | 9,4 | 0,00195 ± 0,0000064 | 0,00788 ± 0,000462 |
| 10.000 | 11,2 | 0,00657 ± 0,000184 | 0,0314 ± 0,00210 |
| 30.000 | 12,6 | 0,0202 ± 0,000270 | 0,147 ± 0,00710 |
| 100.000 | 15,2 | 0,0699 ± 0,000864 | 1,49 ± 0,152 |
| 300.000 | 15,8 | 0,255 ± 0,0108 | 6,03 ± 0,351 |

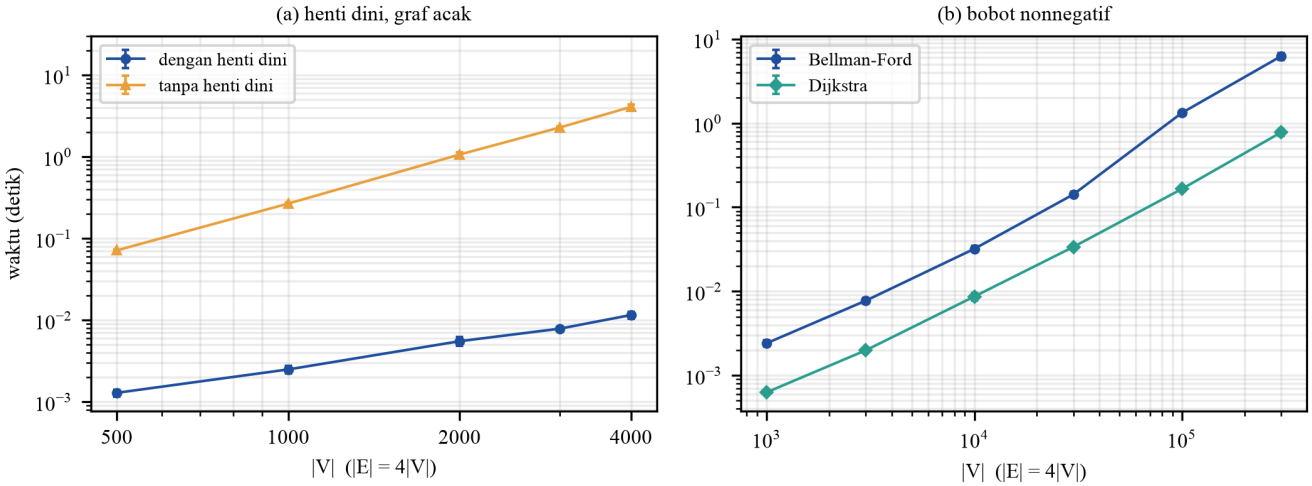
*Cara membaca grafik.* Gambar 3 dan Gambar 4 memakai skala log-log, yaitu kedua sumbunya berskala logaritma. Pada skala ini, rumus waktu $c \cdot |V|^k$ tampil sebagai garis lurus, dan kemiringan garis itu sama dengan k. Nilai k ini disebut *pangkat pertumbuhan*. Pangkat 1 berarti graf 2× lebih besar membuat waktunya 2× lebih lama. Pangkat 2 berarti 4× lebih lama. Jadi untuk mencocokkan hasil dengan teori, cukup bandingkan pangkatnya.

Gambar 3 menyandingkan ketiga keluarga dengan kurva teori (garis putus-putus): $c \cdot |V|^2$ untuk kasus terburuk, $c \cdot |V|$ untuk kasus terbaik, dan $c \cdot (r + 1) \cdot |E|$ untuk graf acak dengan r hasil ukur. Konstanta c hanya menggeser garis naik atau turun. Nilainya dipilih dari data, jadi yang dinilai adalah pangkatnya, bukan nilai mutlaknya. Untuk graf acak, c dipilih dari tiga ukuran terkecil, sehingga penyimpangan pada graf besar terlihat jelas.




Gambar 3. Waktu eksekusi Bellman-Ford buatan sendiri pada tiga keluarga graf, skala log-log, beserta kurva teoretis.

Gambar 4 memuat dua pembandingan: (a) versi dengan dan tanpa henti dini pada graf acak, dan (b) Bellman-Ford terhadap Dijkstra pada bobot nonnegatif.




Gambar 4. Pembandingan dengan versi tanpa penghentian dini dan dengan Dijkstra, skala log-log.

Tabel XI memuat waktu per pemeriksaan sisi, yaitu waktu dibagi $(r + 1) \cdot |E|$. Angka ini menunjukkan seberapa cepat satu langkah dikerjakan, terlepas dari jumlah langkahnya.

TABEL XI. WAKTU PER PEMERIKSAAN SISI (NANODETIK)

| Keluarga | 1.000 | 3.000 | 10.000 | 30.000 | 100.000 | 300.000 |
|---|---|---|---|---|---|---|
| Terbaik | 56 | 54 | 55 | 56 | 58 | 71 |
| Acak | 67 | 63 | 64 | 90 | 230 | 299 |
| Terburuk | 400: 78 | 800: 66 | 1.600: 67 | 3.200: 61 | 6.400: 62 | 12.800: 61 |

Baris terburuk memuat |V| dan nilainya, karena ukurannya berbeda (400 sampai 12.800).

Tabel XII merangkum pangkat pertumbuhan semua garis. Kolom "tiga terbesar" dihitung hanya dari tiga ukuran terbesar, untuk melihat apakah pangkatnya berubah ketika graf makin besar.

TABEL XII. PANGKAT PERTUMBUHAN WAKTU TERHADAP |V| (REGRESI LOG-LOG)

| Keluarga | Algoritma | Teori | Semua titik | Tiga terbesar |
|---|---|---|---|---|
| Terburuk | BF milik sendiri | 2 | 1,94 | 2,00 |
| Terbaik | BF milik sendiri | 1 | 1,03 | 1,10 |
| Acak | BF milik sendiri | 1 sampai 2 | 1,40 | 1,62 |
| Henti dini | dengan henti dini | 1 sampai 2 | 1,05 | 1,05 |
| Henti dini | tanpa henti dini | 2 | 1,95 | 1,93 |
| Bobot nonnegatif | Dijkstra heapq | sedikit di atas 1 | 1,25 | 1,36 |
| Bobot nonnegatif | BF milik sendiri | 1 sampai 2 | 1,40 | 1,65 |

## F. *Pembahasan*

**Kasus terburuk dan terbaik cocok dengan teori.**

- Rantai terbalik: r = |V| − 1 di semua ukuran, dengan pangkat 1,94. Teorinya 2, karena dengan |E| = 4|V|, Θ(|V|·|E|) sama dengan Θ(|V|²).
- Rantai searah: r = 2 di semua ukuran, dengan pangkat 1,03. Teorinya 1.
- Pada kedua keluarga, waktu per pemeriksaan sisi hampir tetap, yaitu 54–78 ns (Tabel XI). Artinya waktu memang sebanding dengan jumlah pemeriksaan sisi, persis seperti rumus T = n + r·m + m di Bagian III-D.

**Graf acak: jumlah putaran sesuai dugaan, tetapi waktunya menyimpang.** Jumlah putaran hanya naik dari 8,2 menjadi 15,8, padahal graf membesar 300 kali. Polanya mengikuti r ≈ 1,42·ln|V| − 1,76 dengan korelasi 0,993, jadi dugaan di Bagian III-D bahwa r tumbuh sebanding log|V| didukung pada rentang ini. Jumlah pemeriksaan sisi juga tumbuh sesuai perkiraan, dengan pangkat 1,11.

Waktu terukurnya justru tumbuh lebih cepat: pangkatnya 1,40, dan 1,62 pada tiga ukuran terbesar. Sebabnya terlihat di Tabel XI. Waktu per pemeriksaan sisi naik dari 67 ns menjadi 299 ns mulai pada 30.000 simpul, sedangkan pada kedua rantai nilainya hampir tetap. Dugaan penulis adalah efek memori. Pada graf acak, urutan sisi diacak, sehingga setiap pemeriksaan mengambil data Python yang letaknya tersebar di memori dan sering tidak tersedia di cache prosesor. Pada rantai, daftar sisinya dibuat berurutan. Dugaan ini cocok dengan pola data, tetapi tidak diuji langsung, misalnya dengan profiler. Jadi analisis di Bagian III-D tepat untuk jumlah langkah, sedangkan waktu nyata pada graf besar juga dipengaruhi memori, yang tidak masuk dalam model.

**Henti dini adalah faktor paling berpengaruh.** Tanpa henti dini, pangkatnya 1,95, sesuai Θ(|V|·|E|). Dengan henti dini, pangkatnya 1,05. Rasio waktunya naik dari 55 kali pada 500 simpul menjadi 353 kali pada 4.000 simpul. Angka ini bisa diperkirakan dari jumlah putaran: versi baku menjalankan |V| putaran termasuk uji siklus, sedangkan versi henti dini menjalankan r + 1. Rasionya sekitar 4.000 / 11,4 ≈ 351, dekat dengan hasil ukur.

**Dijkstra lebih cepat pada bobot nonnegatif, tetapi selisihnya tidak besar.** Bellman-Ford 3,7 sampai 8,0 kali lebih lambat, dan selisihnya membesar pada graf besar. Lonjakan dari 4,2 kali pada 30.000 simpul menjadi 8,0 kali pada 100.000 simpul terjadi pada ukuran yang sama dengan naiknya waktu per pemeriksaan Bellman-Ford di Tabel XI. Selisih ini jauh lebih kecil daripada perkiraan kasus terburuk di Bagian III-E (|V| lawan log|V|), karena pada graf acak Bellman-Ford hanya memerlukan sekitar log|V| putaran. Pangkat Dijkstra 1,25, sedikit di atas 1, sesuai dengan faktor log|V| pada O((|V| + |E|) log|V|). Bagaimanapun, Dijkstra tidak bisa dipakai bila ada bobot negatif (Bagian IV-C).

**Keterbatasan.** Simpangan baku kecil dibandingkan perbedaan antarkeluarga: koefisien variasi terbesar 18% untuk Bellman-Ford dan 11% untuk Dijkstra. Hasil ini dibatasi beberapa hal:

- Pengukuran dilakukan pada satu mesin dan satu implementasi Python, jadi angka 60–70 ns per pemeriksaan tidak berlaku di lingkungan lain.
- Hanya satu kepadatan graf (|E| = 4|V|) dan bobot bulat yang diuji, dan grafnya buatan program.
- Lima ulangan cukup untuk menghitung simpangan baku, tetapi kurang untuk selang kepercayaan.
- Konstanta c dipilih dari data, jadi yang diuji adalah bentuk kurva dan pangkatnya.

## V. Kesimpulan

### A. *Kesimpulan*

Kesimpulan berikut menjawab tiga tujuan pada Bagian I-C.

(1) **Cara kerja, kebenaran, dan kompleksitas.** Bellman-Ford memeriksa semua sisi berulang kali. Setelah putaran ke-k, setiap simpul yang jalur termurahnya berisi paling banyak k sisi sudah bernilai benar (Klaim 1). Jadi |V| − 1 putaran cukup untuk urutan sisi apa pun. Satu putaran tambahan melaporkan siklus negatif jika dan hanya jika ada siklus negatif yang terjangkau dari sumber (Klaim 2). Waktunya sama dengan banyak putaran dikali banyak sisi: Θ(|V|·|E|) untuk versi baku dan kasus terburuk, dan Θ(|V| + |E|) untuk kasus terbaik dengan henti dini. Memori tambahannya Θ(|V|).

(2) **Implementasi dan pengujian.** Inti program ditulis dari nol dalam Python tanpa pustaka. Keempat kasus uji (tiga kasus kecil dan satu studi kasus) lulus, dan hasilnya sama dengan scipy pada 1.000 graf acak (0 selisih). Studi kasus menunjukkan bahwa Dijkstra klasik salah bila ada bobot negatif, dan bahwa urutan sisi hanya mengubah jumlah putaran, bukan hasil.

(3) **Pengukuran terhadap teori.** Kasus terburuk dan terbaik cocok dengan teori, dengan pangkat pertumbuhan 1,94 dan 1,03 (teori 2 dan 1). Pada graf acak, jumlah putaran tumbuh sebanding log|V| seperti dugaan. Namun waktu nyatanya melambat pada graf besar, dan ini diduga karena efek memori. Henti dini membuat graf acak 353 kali lebih cepat pada 4.000 simpul. Dijkstra 3,7 sampai 8,0 kali lebih cepat pada bobot nonnegatif, tetapi tidak bisa dipakai bila ada bobot negatif.

### B. *Kelebihan dan Keterbatasan*

Kelebihan Bellman-Ford:

- Sederhana: hanya perlu daftar sisi dan dua larik.
- Menerima bobot negatif dan mendeteksi siklus negatif yang terjangkau dari sumber.
- Dengan henti dini, cepat pada graf yang jalur termurahnya pendek. Waktunya mendekati sebanding dengan |E|.

Keterbatasannya:

- Kasus terburuknya Θ(|V|·|E|), sehingga pada graf besar berbobot nonnegatif kalah cepat dari Dijkstra. Versi bakunya selalu memakan waktu sebesar ini.
- Siklus negatif yang tidak terjangkau dari sumber tidak terdeteksi.

Hasil eksperimen di laporan ini juga terbatas pada satu mesin, satu implementasi Python, satu kepadatan graf, dan graf buatan program (Bagian IV-F).

### C. Peluang Pengembangan

- Varian yang mengurangi jumlah relaksasi, yaitu pengurutan sisi menurut Yen [12] dan pengacakan urutan menurut Bannister dan Eppstein [14], dapat diukur dengan kode eksperimen yang sama.
- Algoritma yang secara teori lebih cepat [8], [9], [10] dapat dibandingkan dengan Bellman-Ford klasik pada bobot bilangan bulat.
- Dugaan efek memori dapat diuji dengan bahasa yang dikompilasi atau dengan alat pengukur cache. Eksperimen juga dapat diperluas ke beberapa kepadatan graf dan ke data nyata, misalnya kurs untuk arbitrase [7].
- Algoritma Johnson [1] dapat dibuat sebagai penerapan langsung Bellman-Ford untuk menghitung jarak antara semua pasangan simpul.

## LAMPIRAN

### *Lampiran A. Tautan Kode dan Cara Menjalankan*

Kode dan data dikumpulkan bersama laporan ini:

- **Bellman_Ford_Sederhana.ipynb**: inti tanpa pustaka, tiga kasus kecil, dan studi kasus
- **Bellman_Ford_Eksperimen.ipynb**: uji silang dan eksperimen waktu
- **README.md**: daftar berkas dan cara menjalankan
- **hasil_eksperimen.csv**, **grafik_waktu_tiga_kasus.png**, **grafik_pembanding.png**: data dan grafik eksperimen
- **Penjelasan_Bellman_Ford.html**: visualisasi interaktif
- **Presentasi_Bellman_Ford.html** dan **Presentasi_Ringkas_Bellman_Ford.html**: halaman presentasi

[ISI tautan repositori atau lokasi pengumpulan.]

Notebook sederhana tidak memerlukan pustaka apa pun. Notebook eksperimen memerlukan Python 3.10 atau lebih baru dengan numpy, matplotlib, dan scipy (scipy hanya kunci jawaban uji silang), dan berjalan sekitar 13 menit pada mesin di Tabel IV. Keduanya dijalankan dengan Restart and Run All. Seed acak tetap 2026; waktu dinding sedikit berbeda pada setiap jalan, sedangkan jumlah putaran dan seluruh jarak selalu sama.

### *Lampiran B. Pemetaan Pseudocode ke Kode*

Setiap baris pseudocode pada Algoritma 1 memiliki pasangan pada inti Python di Lampiran D. Tabel berikut dipakai untuk menelusuri kode baris demi baris.

TABEL XIII. PEMETAAN BARIS PSEUDOCODE KE INTI PYTHON

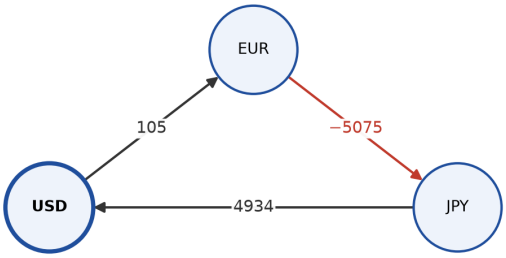
| Baris | Pseudocode | Kode Python (fungsi) |
|---|---|---|
| 1–2 | d[v] ← ∞, π[v] ← NIL; d[s] ← 0 | `dist, pred = [INF] * n, [None] * n; dist[src] = 0` (inisialisasi) |
| 3 | ulangi \|V\| − 1 kali | `for _ in range(n - 1):` (bellman_ford) |
| 4 | untuk setiap sisi (u, v) | `for u, v, w in edges:` (satu_putaran) |
| 5 | jika d[u] < ∞ dan d[u] + w < d[v] | `if dist[u] != INF and dist[u] + w < dist[v]:` (relaksasi) |
| 6 | d[v] ← d[u] + w; π[v] ← u | `dist[v] = dist[u] + w; pred[v] = u` (relaksasi) |
| – | penghentian dini (tambahan) | `if berhenti_dini and not ada_perubahan: break` |
| 7–8 | uji siklus negatif | fungsi ada_siklus_negatif |
| 9 | kembalikan (d, π, ...) | `return dist, pred, ada_siklus_negatif(edges, dist)` |

### *Lampiran C. Jejak Lengkap Kasus Uji dan Studi Kasus*

Tabel berikut dihasilkan langsung oleh fungsi bellman_ford pada notebook sederhana dengan parameter jejak. Pada studi kasus, kolom memakai singkatan Mks (Makassar), Mrs (Maros), Pkp (Pangkep), Brr (Barru), Ppr (Parepare), Sdr (Sidrap), Pin (Pinrang), dan Enr (Enrekang).

TABEL XIV. PEMERIKSAAN SISI SATU PER SATU PADA KASUS KURIR (GAMBAR 1), TERMASUK PUTARAN UJI

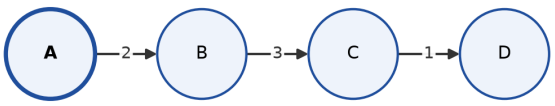
| Put. | Sisi | d[u] + w | d[v] lama | Hasil |
|---|---|---|---|---|
| 1 | S1 Gudang→Toko A | 4 | ∞ | d ← 4 |
| 1 | S2 Gudang→Toko B | 5 | ∞ | d ← 5 |
| 1 | S3 Toko B→Toko A | 2 | 4 | d ← 2 |
| 1 | S4 Toko A→Pelanggan | 4 | ∞ | d ← 4 |
| 2 | S1 Gudang→Toko A | 4 | 2 | tetap |
| 2 | S2 Gudang→Toko B | 5 | 5 | tetap |
| 2 | S3 Toko B→Toko A | 2 | 2 | tetap |
| 2 | S4 Toko A→Pelanggan | 4 | 4 | tetap |
| uji | S1 Gudang→Toko A | 4 | 2 | aman |
| uji | S2 Gudang→Toko B | 5 | 5 | aman |
| uji | S3 Toko B→Toko A | 2 | 2 | aman |
| uji | S4 Toko A→Pelanggan | 4 | 4 | aman |




Gambar 5. Graf kasus 2, segitiga tukar uang. Bobot = −1000·ln(kurs), jumlah keliling −36.

TABEL XV. PEMERIKSAAN SISI SATU PER SATU PADA KASUS 2

| Put. | Sisi | d[u] + w | d[v] lama | Hasil |
|---|---|---|---|---|
| 1 | USD→EUR | 105 | ∞ | d ← 105 |
| 1 | EUR→JPY | −4970 | ∞ | d ← −4970 |
| 1 | JPY→USD | −36 | 0 | d ← −36 |
| 2 | USD→EUR | 69 | 105 | d ← 69 |
| 2 | EUR→JPY | −5006 | −4970 | d ← −5006 |
| 2 | JPY→USD | −72 | −36 | d ← −72 |
| uji | USD→EUR | 33 | 69 | siklus negatif |




Gambar 6. Graf kasus 3, rantai A → B → C → D. Daftar terbalik memeriksa C → D, B → C, lalu A → B.

TABEL XVI. PEMERIKSAAN SISI SATU PER SATU PADA KASUS 3, DAFTAR TERBALIK

| Put. | Sisi | d[u] + w | d[v] lama | Hasil |
|---|---|---|---|---|
| 1 | C→D | – | ∞ | dilewati |
| 1 | B→C | – | ∞ | dilewati |
| 1 | A→B | 2 | ∞ | d ← 2 |
| 2 | C→D | – | ∞ | dilewati |
| 2 | B→C | 5 | ∞ | d ← 5 |
| 2 | A→B | 2 | 2 | tetap |
| 3 | C→D | 6 | ∞ | d ← 6 |
| 3 | B→C | 5 | 5 | tetap |
| 3 | A→B | 2 | 2 | tetap |
| uji | C→D | 6 | 6 | aman |
| uji | B→C | 5 | 5 | aman |
| uji | A→B | 2 | 2 | aman |

TABEL XVII. JEJAK D PADA KASUS 2, SEGITIGA TUKAR UANG (SIKLUS −36)

| Putaran | USD | EUR | JPY |
|---|---|---|---|
| Awal | 0 | ∞ | ∞ |
| 1 | −36 | 105 | −4970 |
| 2 | −72 | 69 | −5006 |

TABEL XVIII. JEJAK D PADA KASUS 3, DAFTAR TERBALIK

| Putaran | A | B | C | D |
|---|---|---|---|---|
| Awal | 0 | ∞ | ∞ | ∞ |
| 1 | 0 | 2 | ∞ | ∞ |
| 2 | 0 | 2 | 5 | ∞ |
| 3 | 0 | 2 | 5 | 6 |

TABEL XIX. JEJAK D PADA KASUS 3, DAFTAR SEARAH

| Putaran | A | B | C | D |
|---|---|---|---|---|
| Awal | 0 | ∞ | ∞ | ∞ |
| 1 | 0 | 2 | 5 | 6 |
| 2 | 0 | 2 | 5 | 6 |

Pada kasus 2, sesudah putaran 2 uji siklus masih menemukan USD → EUR: −72 + 105 = 33 < 69, sehingga siklus negatif dilaporkan. Catatan sumber sendiri sudah turun ke −36 pada putaran 1, tanda bahwa ada jalan kembali ke sumber yang lebih murah dari nol.

TABEL XX. JEJAK D PADA STUDI KASUS DISTRIBUSI

| Putaran | Mks | Mrs | Pkp | Brr | Ppr | Sdr | Pin | Enr |
|---|---|---|---|---|---|---|---|---|
| Awal | 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| 1 | 0 | 40 | 90 | ∞ | ∞ | ∞ | ∞ | ∞ |
| 2 | 0 | 40 | 90 | 70 | 145 | 200 | 205 | ∞ |
| 3 | 0 | 40 | 90 | 70 | 145 | 175 | 205 | 245 |
| 4 | 0 | 40 | 90 | 70 | 145 | 175 | 205 | 245 |

*Lampiran D. Kode Inti Lengkap*

Isi sel inti pada notebook Bellman_Ford_Sederhana.ipynb, tanpa perubahan. Sel yang sama dipakai oleh notebook eksperimen.

```
INF = float("inf")



def inisialisasi(n, src):
    """Pseudocode baris 1-2: semua jarak INF kecuali sumber 0, semua pendahulu None."""
    dist = [INF] * n          # jarak sementara d[v]; INF = belum ada jalur yang diketahui
    pred = [None] * n         # pendahulu pi[v]; None = belum ada
    dist[src] = 0             # jarak sumber ke dirinya sendiri adalah 0
    return dist, pred



def relaksasi(u, v, w, dist, pred):
    """Pseudocode baris 5-6: perbarui d[v] bila lewat u lebih murah. Kembalikan True bila ada pembaruan."""
    # Sisi dari simpul yang belum terjangkau tidak boleh dipakai. Dengan INF float, INF + w tetap INF,
    # tetapi bila INF diganti bilangan besar biasa (mis. 10**9), jarak palsu bisa muncul.
    if dist[u] != INF and dist[u] + w < dist[v]:   # tanda '<' (bukan '<='): nilai sama bukan perbaikan
        dist[v] = dist[u] + w
        pred[v] = u
        return True
    return False



def satu_putaran(edges, dist, pred):
    """Pseudocode baris 4: periksa semua sisi sekali, sesuai urutan di edges.
    dist dan pred diubah di tempat, jadi pembaruan di tengah putaran langsung terlihat oleh sisi berikutnya."""
    ada_perubahan = False
    for u, v, w in edges:
        if relaksasi(u, v, w, dist, pred):
            ada_perubahan = True
    return ada_perubahan



def ada_siklus_negatif(edges, dist):
    """Pseudocode baris 7-8: putaran ke-|V|. Masih ada sisi yang bisa memperbarui d berarti ada siklus
    negatif yang terjangkau dari sumber. Fungsi ini tidak mengubah dist."""
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            return True
    return False



def bellman_ford(n, edges, src, jejak=None, berhenti_dini=True):
    """Bellman-Ford: jarak terpendek dari satu sumber, bobot negatif diperbolehkan.
```

```
    Masukan : n     jumlah simpul (bernomor 0..n-1)
              edges daftar (u, v, w) untuk sisi berarah u -> v berbobot w
              src   simpul sumber
    Keluaran: (dist, pred, ada_siklus_negatif)
              dist[v] jarak terpendek (INF bila tak terjangkau)
              pred[v] simpul sebelum v pada jalur terpendek (None bila tidak ada)
    jejak   : opsional. Bila bukan None, diisi salinan dist: sekali setelah inisialisasi, lalu sekali di
              akhir tiap putaran. Dipakai untuk mencetak tabel jarak per putaran.
    berhenti_dini : bila True, perulangan berhenti begitu satu putaran tidak mengubah apa pun.
    """
    dist, pred = inisialisasi(n, src)                    # baris 1-2
    if jejak is not None:
        jejak.append(dist[:])
    for _ in range(n - 1):                               # baris 3: paling banyak |V| - 1 putaran
        ada_perubahan = satu_putaran(edges, dist, pred)  # baris 4-6
        if jejak is not None:
            jejak.append(dist[:])
        if berhenti_dini and not ada_perubahan:          # penghentian dini (aman: putaran berikutnya identik)
            break
    return dist, pred, ada_siklus_negatif(edges, dist)   # baris 7-9



def rekonstruksi_jalur(pred, src, target):
    """Susun jalur src -> target dengan menelusuri pred mundur. Kembalikan [] bila target tidak terjangkau."""
    jalur = []
    sekarang = target
    langkah = 0
    while sekarang is not None and langkah <= len(pred):   # batas langkah = pengaman bila pred berlingkaran
        jalur.append(sekarang)
        sekarang = pred[sekarang]
        langkah += 1
    if sekarang is not None:                               # berhenti karena batas langkah: pred berlingkaran
        return []
    jalur.reverse()
    return jalur if jalur and jalur[0] == src else []      # simpul tak terjangkau berujung bukan di src
```

*Lampiran E. Data Mentah Eksperimen*

Isi berkas hasil_eksperimen.csv: rata-rata dan simpangan baku waktu dari lima ulangan, serta rata-rata jumlah putaran r pada versi dengan penghentian dini. Baris BF scipy dicatat oleh notebook eksperimen, tetapi tidak dibahas, karena scipy hanya dipakai sebagai kunci jawaban.

TABEL XXI. DATA MENTAH EKSPERIMEN

| Keluarga | Algoritma | \|V\| | \|E\| | r | Rata-rata (s) | Simpangan baku (s) |
|---|---|---|---|---|---|---|
| terburuk | BF milik sendiri | 400 | 1.600 | 399,0 | 0,0498 | 0,00874 |
| terburuk | BF scipy | 400 | 1.600 | 399,0 | 0,00122 | 0,0000471 |
| terburuk | BF milik sendiri | 800 | 3.200 | 799,0 | 0,168 | 0,00716 |
| terburuk | BF scipy | 800 | 3.200 | 799,0 | 0,00478 | 0,000222 |
| terburuk | BF milik sendiri | 1.600 | 6.400 | 1599,0 | 0,684 | 0,0358 |
| terburuk | BF scipy | 1.600 | 6.400 | 1599,0 | 0,0218 | 0,00363 |
| terburuk | BF milik sendiri | 3.200 | 12.800 | 3199,0 | 2,51 | 0,0399 |
| terburuk | BF scipy | 3.200 | 12.800 | 3199,0 | 0,0883 | 0,00105 |
| terburuk | BF milik sendiri | 6.400 | 25.600 | 6399,0 | 10,1 | 0,124 |
| terburuk | BF scipy | 6.400 | 25.600 | 6399,0 | 0,393 | 0,00515 |
| terburuk | BF milik sendiri | 12.800 | 51.200 | 12799,0 | 40,0 | 0,506 |
| terburuk | BF scipy | 12.800 | 51.200 | 12799,0 | 1,65 | 0,0209 |
| terbaik | BF milik sendiri | 1.000 | 4.000 | 2,0 | 0,000678 | 0,0000020 |
| terbaik | BF scipy | 1.000 | 4.000 | 2,0 | 0,00705 | 0,0000375 |
| terbaik | BF milik sendiri | 3.000 | 12.000 | 2,0 | 0,00195 | 0,0000064 |
| terbaik | BF scipy | 3.000 | 12.000 | 2,0 | 0,0812 | 0,00192 |
| terbaik | BF milik sendiri | 10.000 | 40.000 | 2,0 | 0,00657 | 0,000184 |
| terbaik | BF scipy | 10.000 | 40.000 | 2,0 | 0,987 | 0,00249 |
| terbaik | BF milik sendiri | 30.000 | 120.000 | 2,0 | 0,0202 | 0,000270 |
| terbaik | BF scipy | 30.000 | 120.000 | 2,0 | 9,37 | 0,0118 |
| terbaik | BF milik sendiri | 100.000 | 400.000 | 2,0 | 0,0699 | 0,000864 |
| terbaik | BF milik sendiri | 300.000 | 1.200.000 | 2,0 | 0,255 | 0,0108 |
| acak | BF milik sendiri | 1.000 | 4.000 | 8,2 | 0,00247 | 0,000240 |
| acak | BF scipy | 1.000 | 4.000 | 8,2 | 0,00710 | 0,0000626 |
| acak | BF milik sendiri | 3.000 | 12.000 | 9,4 | 0,00788 | 0,000462 |
| acak | BF scipy | 3.000 | 12.000 | 9,4 | 0,0801 | 0,000803 |
| acak | BF milik sendiri | 10.000 | 40.000 | 11,2 | 0,0314 | 0,00210 |
| acak | BF scipy | 10.000 | 40.000 | 11,2 | 0,980 | 0,00439 |
| acak | BF milik sendiri | 30.000 | 120.000 | 12,6 | 0,147 | 0,00710 |
| acak | BF scipy | 30.000 | 120.000 | 12,6 | 9,45 | 0,0192 |
| acak | BF milik sendiri | 100.000 | 400.000 | 15,2 | 1,49 | 0,152 |
| acak | BF milik sendiri | 300.000 | 1.200.000 | 15,8 | 6,03 | 0,351 |
| ablasi | BF milik sendiri | 500 | 2.000 | 8,0 | 0,00129 | 0,000122 |
| ablasi | BF tanpa henti dini | 500 | 2.000 | 8,0 | 0,0714 | 0,00130 |
| ablasi | BF milik sendiri | 1.000 | 4.000 | 8,2 | 0,00249 | 0,000250 |
| ablasi | BF tanpa henti dini | 1.000 | 4.000 | 8,2 | 0,267 | 0,00740 |
| ablasi | BF milik sendiri | 2.000 | 8.000 | 9,4 | 0,00552 | 0,000659 |
| ablasi | BF tanpa henti dini | 2.000 | 8.000 | 9,4 | 1,06 | 0,0768 |
| ablasi | BF milik sendiri | 3.000 | 12.000 | 9,4 | 0,00786 | 0,000426 |
| ablasi | BF tanpa henti dini | 3.000 | 12.000 | 9,4 | 2,28 | 0,0791 |

| Keluarga | Algoritma | \|V\| | \|E\| | r | Rata-rata (s) | Simpangan baku (s) |
|---|---|---|---|---|---|---|
| ablasi | BF milik sendiri | 4.000 | 16.000 | 10,4 | 0,0115 | 0,00113 |
| ablasi | BF tanpa henti dini | 4.000 | 16.000 | 10,4 | 4,07 | 0,292 |
| dijkstra | BF milik sendiri | 1.000 | 4.000 | 8,2 | 0,00242 | 0,000201 |
| dijkstra | Dijkstra heapq | 1.000 | 4.000 | 8,2 | 0,000630 | 0,0000280 |
| dijkstra | BF milik sendiri | 3.000 | 12.000 | 9,6 | 0,00775 | 0,000601 |
| dijkstra | Dijkstra heapq | 3.000 | 12.000 | 9,6 | 0,00199 | 0,0000143 |
| dijkstra | BF milik sendiri | 10.000 | 40.000 | 11,4 | 0,0320 | 0,00240 |
| dijkstra | Dijkstra heapq | 10.000 | 40.000 | 11,4 | 0,00868 | 0,000926 |
| dijkstra | BF milik sendiri | 30.000 | 120.000 | 12,8 | 0,142 | 0,00964 |
| dijkstra | Dijkstra heapq | 30.000 | 120.000 | 12,8 | 0,0338 | 0,000914 |
| dijkstra | BF milik sendiri | 100.000 | 400.000 | 13,8 | 1,33 | 0,0510 |
| dijkstra | Dijkstra heapq | 100.000 | 400.000 | 13,8 | 0,165 | 0,00456 |
| dijkstra | BF milik sendiri | 300.000 | 1.200.000 | 16,0 | 6,24 | 0,565 |
| dijkstra | Dijkstra heapq | 300.000 | 1.200.000 | 16,0 | 0,775 | 0,0766 |

### *Lampiran F. Pelaporan Penggunaan AI*

Kode, analisis, eksperimen, dan naskah laporan ini disusun dengan bantuan asisten AI Claude (Anthropic). Penulis bertanggung jawab atas seluruh isi.

## DAFTAR PUSTAKA


[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] A. Shimbel, "Structure in communication nets," in *Proc. Symp. Inf. Netw.*, New York, NY, USA: Polytechnic Press of the Polytechnic Inst. of Brooklyn, 1955, pp. 199–203.

[3] L. R. Ford, Jr., "Network flow theory," RAND Corp., Santa Monica, CA, USA, Paper P-923, Aug. 1956. [Online]. Available: https://apps.dtic.mil/sti/pdfs/AD0422842.pdf

[4] R. Bellman, "On a routing problem," *Quart. Appl. Math.*, vol. 16, no. 1, pp. 87–90, 1958, doi: 10.1090/qam/102435.

[5] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. Theory of Switching, Part II*, Cambridge, MA, USA, Apr. 1957. Cambridge, MA, USA: Harvard Univ. Press, 1959, pp. 285–292.

[6] C. L. Hedrick, "Routing Information Protocol," Internet Eng. Task Force, RFC 1058, Jun. 1988. [Online]. Available: https://www.rfc-editor.org/info/rfc1058

[7] I. Akouaouch and A. Bouayad, "An innovative approach to identifying triangular arbitrage opportunities in financial markets using the Bellman-Ford algorithm," *Bull. Elect. Eng. Informat.*, vol. 15, no. 3, pp. 2579–2594, Jun. 2026, doi: 10.11591/eei.v15i3.10817.

[8] A. Bernstein, D. Nanongkai, and C. Wulff-Nilsen, "Negative-weight single-source shortest paths in near-linear time," in *Proc. IEEE 63rd Annu. Symp. Found. Comput. Sci. (FOCS)*, Denver, CO, USA, 2022, pp. 600–611, doi: 10.1109/FOCS54457.2022.00063.

[9] J. T. Fineman, "Single-source shortest paths with negative real weights in $\tilde{O}(mn^{8/9})$ time," in *Proc. 56th Annu. ACM Symp. Theory Comput. (STOC)*, Vancouver, BC, Canada, 2024, pp. 3–14, doi: 10.1145/3618260.3649614.

[10] Y. Huang, P. Jin, and K. Quanrud, "Faster single-source shortest paths with negative real weights via proper hop distance," in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2025, pp. 5239–5244, doi: 10.1137/1.9781611978322.178.

[11] V. Ashvinkumar *et al.*, "Parallel, distributed, and quantum exact single-source shortest paths with negative edge weights," in *Proc. 32nd Annu. Eur. Symp. Algorithms (ESA)*, Leibniz Int. Proc. Informat. (LIPIcs), vol. 308, 2024, Art. no. 13, pp. 13:1–13:15, doi: 10.4230/LIPIcs.ESA.2024.13.

[12] J. Y. Yen, "An algorithm for finding shortest routes from all source nodes to a given destination in general networks," *Quart. Appl. Math.*, vol. 27, no. 4, pp. 526–530, 1970, doi: 10.1090/qam/253822.

[13] J. Hu and L. Kozma, "Non-adaptive Bellman–Ford: Yen's improvement is optimal," 2024, *arXiv:2402.10343* (versi diperbarui Okt. 2026).

[14] M. J. Bannister and D. Eppstein, "Randomized speedup of the Bellman–Ford algorithm," in *Proc. 9th Workshop Analytic Algorithmics Combinatorics (ANALCO)*, Kyoto, Japan, 2012, pp. 41–47, doi: 10.1137/1.9781611973020.6.